Deep Learning - COSC2779

Neural Network Optimization

A/Prof. Ruwan Tennakoon

RMIT UNIVERSITY

Reference: *Chapter 7,8: Ian Goodfellow et. al., "Deep Learning", MIT Press, 2016.*

Part 1: Optimization Techniques

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

Part 2: Regularization

The **Task** can be expressed an unknown target function:

$$\mathbf{y} = f(\mathbf{x})$$

ML finds a Hypothesis (model), $h(\cdot)$, from hypothesis space $\mathcal{H}$, which approximates the unknown target function.

$$\hat{\mathbf{y}} = h^*(\mathbf{x}) \approx f(\mathbf{x})$$

The **Experience** is typically a data set, $\mathcal{D}$, of values

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(i)}, f\left(\mathbf{x}^{(i)}\right) \right) \right\}_{i=1}^{N}$$

∗Assume supervised learning for now

The **Performance** is typically numerical measure that determines how well the hypothesis matches the experience.

Last week we discussed about *a representation* of Hypothesis (model), $h(\cdot)$.

$$h(\mathbf{x}) = h^{(3)}\left(h^{(2)}\left(h^{(1)}(\mathbf{x})\right)\right)$$

This week: How can we find the optimal hypothesis $h^*(\mathbf{x})$?

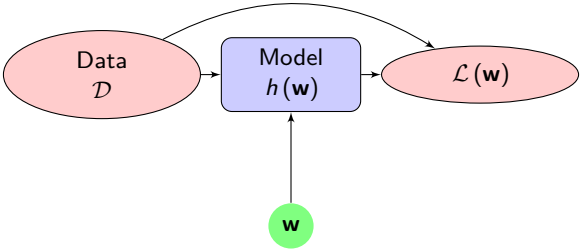
Objectives for this Lecture

- Explore techniques that can be used to find the optimal hypothesis in a NN.
- Understand the optimization techniques so that we can come up with the "best approach for a problem" in a way that is better than random or exhaustive search of the applicable techniques in the deep learning toolbox.
- Start understanding how to do evidence based debugging when the model is not learning.
- Applicable to all NN types, not just feed-forward.

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

;



We would like to finding the parameters, $\mathbf{w}$, of a neural network that reduce a cost function $\mathcal{L}(\mathbf{w})$.

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\ \mathcal{L}(\mathbf{w})$$

We would like to finding the parameters, $\mathbf{w}$, of a neural network that reduce a cost function $\mathcal{L}(\mathbf{w})$.

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\ \mathcal{L}(\mathbf{w})$$

The cost function usually consists of two parts. The loss and regularization terms.

$$\mathcal{L}(\mathbf{w}) = \underbrace{\mathbb{E}_{(\mathbf{x},y)\sim p_{data}} \mathrm{L}\left(y, h(\mathbf{x};\mathbf{w})\right)}_{\text{Risk}} + \lambda \underbrace{\mathrm{R}(\mathbf{w})}_{\text{Regularization}}$$

Here p_{data} is the data-generating distribution. $\mathrm{L}()$ is some function that quantify the deviation from expected result.

This is an optimization problem. But, in ML, We do not know p_{data}, Therefore we cannot minimize **risk**.

∗Out main attention in these slides is given to supervised learning.

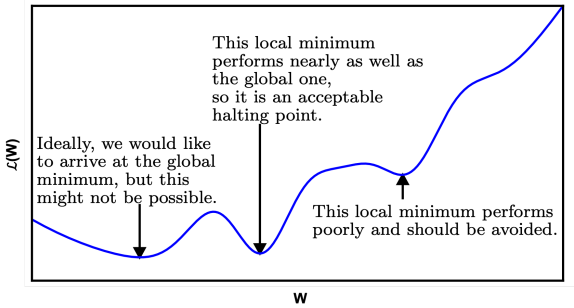
In ML, We do not know p_{data}. Therefore we minimize the empirical risk. Minimize the expected loss on the training set.

$$\mathcal{L}(\mathbf{w}) = \underbrace{\mathbb{E}_{(\mathbf{x},y)\sim\hat{p}_{data}} \mathrm{L}(y, h(\mathbf{x};\mathbf{w}))}_{\text{Empirical Risk}} + \lambda \mathrm{R}(\mathbf{w})$$

Here $\hat{p}_{data}$ is the training data distribution.

$$\mathbb{E}_{(\mathbf{x},y)\sim\hat{p}_{data}} \mathrm{L}(y, h(\mathbf{x};\mathbf{w})) = \frac{1}{N}\sum_{i=1}^{N} \mathrm{L}\left(y^{(i)}, h\left(\mathbf{x}^{(i)};\mathbf{w}\right)\right)$$

The training process based on minimizing this average training error is known as **empirical risk minimization**.

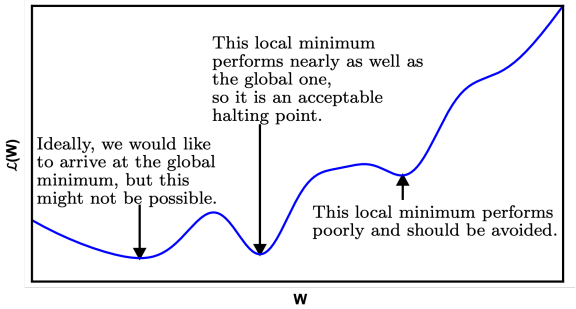


The derivative of the loss function is zero ($\frac{\partial \mathcal{L}(w)}{\partial w} = 0$) at any critical point in the loss function.

- A **local minimum** is a point where $\mathcal{L}(w)$ is lower than at all neighboring points.
- A point that obtains the absolute lowest value of $\mathcal{L}(w)$ is a **global minimum**.

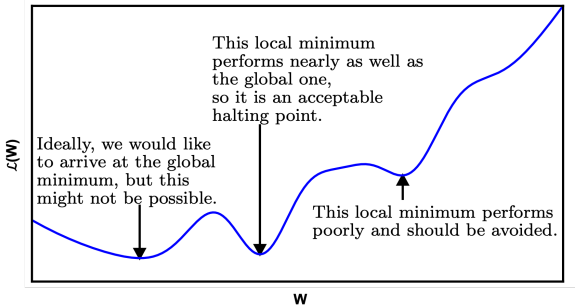
If the loss is **convex** we have only one minimum - The global minimum.

Finding the Minimum



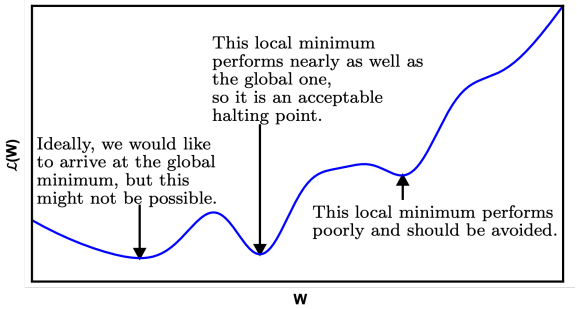
- Random search - Bad idea.

Finding the Minimum



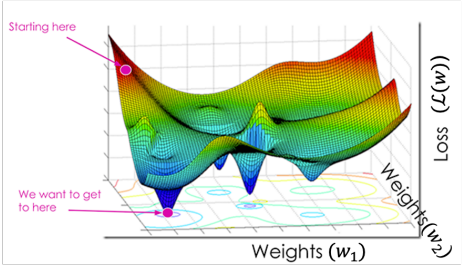
- Random search - Bad idea.
- Solve $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 0$ and find a closed form solution. Not applicable for many cases.

Finding the Minimum



- Random search - Bad idea.
- Solve $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = 0$ and find a closed form solution. Not applicable for many cases.
- Guided Search - Apply iterative method - Gradient decent: $\mathbf{w}^{[t]} = \mathbf{w}^{[t-1]} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

Gradient descent



Note that $\mathbf{w} = [w_0, w_1, \cdots, w_m]$

Algorithm 1: Basic Gradient Decent

Result: Final Weights $\mathbf{w}$

Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$;

while *not converged* **do**

  Compute Gradients, $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$;

  Update weights, $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$;

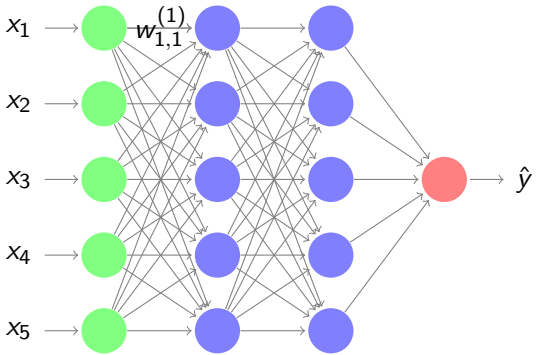
end

α is the learning rate.

How can we efficiently apply gradient decent to Neural networks?

Outline

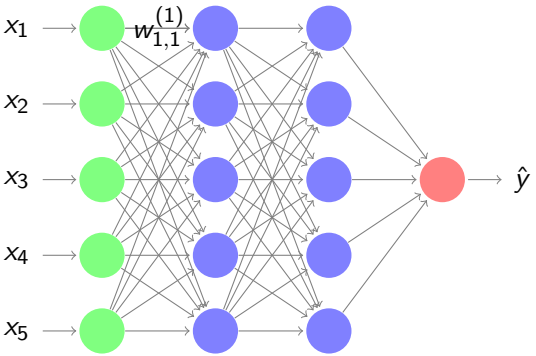
1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization



The gradient, $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$, is simply the vector of partial derivatives in each dimension.

How can we calculate $\frac{\partial}{\partial w_{1,1}^{(1)}} \mathcal{L}(\mathbf{w})$?

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \left[\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}_{1,1}^{(1)}}, \frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}_{1,2}^{(1)}}, \cdots \right]$$



The gradient, $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$, is simply the vector of partial derivatives in each dimension.

How can we calculate $\frac{\partial}{\partial w_{1,1}^{(1)}} \mathcal{L}(\mathbf{w})$?

Numerical derivatives (finite difference approximation):

$$\frac{\partial}{\partial w_{1,1}^{(1)}} \mathcal{L}(\mathbf{w}) = \lim_{\delta \to 0} \frac{\mathcal{L}\left(w_{1,1}^{(1)} + \delta\right) - \mathcal{L}\left(w_{1,1}^{(1)}\right)}{\delta}$$

∗Other elements of $\mathbf{w}$ will remain constant.

How about $\frac{\partial}{\partial w_{1,2}^{(1)}} \mathcal{L}(\mathbf{w})$?

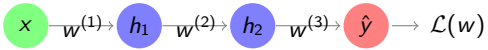
This looks expensive, if we have millions of weights.

Back-Propagation is a computationally efficient method to calculate derivatives.

Repeated application of the chain-rule.

Lets have a look at the intuition behind back-prop using a simple neural network.

A good explanation of back-prop is in Calculus on Computational Graphs: Backpropagation



$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w^{(3)}} = \frac{\partial \mathcal{L}(\mathbf{w})}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(3)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w^{(2)}} = \underbrace{\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \hat{y}}} \times \frac{\partial \hat{y}}{\partial h_2} \times \frac{\partial h_2}{\partial w^{(2)}}$$

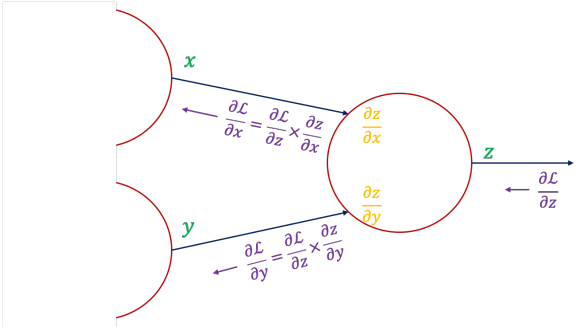
$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w^{(1)}} = \underbrace{\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_2}} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial w^{(1)}}$$

Back-Propagation is a computationally efficient method to calculate derivatives.

Repeated application of the chain-rule.

Lets have a look at the intuition behind back-prop using a simple neural network.

A good explanation of back-prop is in Calculus on Computational Graphs: Backpropagation



Implementation - PyTorch

In PyTorch, the autograd tracks operations on your tensors to dynamically build a computational graph. You only need to define the forward pass.
While we manually updated the weights in the example above, PyTorch provides built-in modules (torch.nn and torch.optim) that manage the optimization steps under the hood.

```python
import torch

# Initialize a random weight and tell PyTorch to track gradients
w = torch.randn(1, requires_grad=True)
lr = 0.001

while True: # loop forever
    # 1. Forward pass (the graph is built automatically here)
    loss = compute_loss(w)  # forward function defined outside

    # 2. Compute gradients (populates w.grad)
    loss.backward()

    # 3. Update weights
    # We use torch.no_grad() so this update step isn't tracked in the
    # computational graph
    with torch.no_grad():
        w -= lr * w.grad

        # CRUCIAL: PyTorch accumulates gradients, so you must manually
        # zero them out
        # after every update step, otherwise they will keep adding up.
        w.grad.zero_()

    # need to check convergence and break loop
```

1. Why cannot we directly minimize the risk in ML?
2. What is the difference between the notations $\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w})$ and $\frac{\partial}{\partial w}\mathcal{L}(\mathbf{w})$?
3. Will gradient decent find the weights that minimize the cost function in deep feed-forward NN?
4. When calculating $\frac{\partial}{\partial w_{1,1}^{(1)}}\mathcal{L}(\mathbf{w})$, what valaues will you use for other weights (e.g. $w_{1,2}^{(1)}$)?

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

Calculating the gradients of the empirical risk.

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \nabla_{\mathbf{w}} \mathbb{E}_{(\mathbf{x},y)\sim \hat{p}_{data}} \mathrm{L}(y, h(\mathbf{x}; \mathbf{w})) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} \mathrm{L}\left(y^{(i)}, h\left(\mathbf{x}^{(i)}; \mathbf{w}\right)\right)$$

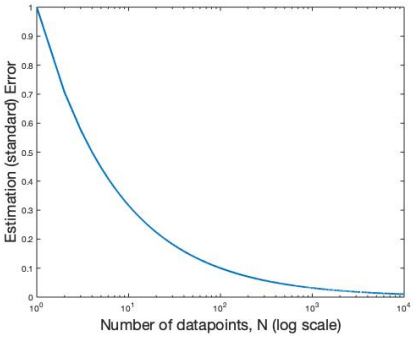
Computing this expectation exactly is **very expensive** because it requires evaluating the model on every example in the entire dataset.

In practice, we can compute these expectations by randomly sampling a small number of examples from the dataset, then taking the average over only those examples.

$$\nabla_{\mathbf{w}} \mathbb{E}_{(\mathbf{x},y)\sim \hat{p}_{data}} \mathrm{L}(y, h(\mathbf{x}; \mathbf{w})) \approx \frac{1}{N_b} \sum_{i=1}^{N_b} \nabla_{\mathbf{w}} \mathrm{L}\left(y^{(i)}, h\left(\mathbf{x}^{(i)}; \mathbf{w}\right)\right)$$

Why does approximate derivatives work (intuition):

- Standard error of the mean estimated with n samples is $\sigma/\sqrt{n}$. Reduction in standard error with the increase in n diminishes with n.



Why does approximate derivatives work (intuition):

- Standard error of the mean estimated with n samples is $\sigma/\sqrt{n}$. Reduction in standard error with the increase in n diminishes with n.
- Redundancy in the training set. Some data-points will result in the same values.
- We just need an approximate direction to step. Will work as long as the gradients are not totally random.

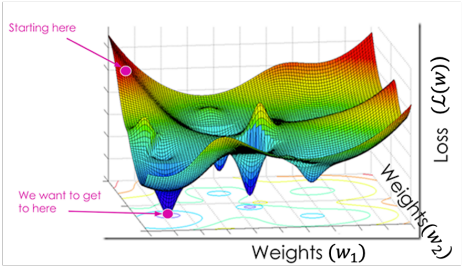
Gradient decent methods:

- **Deterministic**: Uses all the training data when calculating gradient for each step.
- **Stochastic** (mini-batch): Use randomly sampled small number of examples from training data when calculating gradient for each step

- Larger batches provide a more accurate estimate of the gradient, but with less than linear returns
- Multi-core architectures are usually underutilized by extremely small batches.
- Amount of memory scales with the batch size.
- When using GPUs, it is common for power of 2 batch sizes to offer better run time.
- Small batches can offer a regularizing effect.

The general inefficiency of batch training for gradient descent learning

We also wish for two subsequent gradient estimates to be independent from each other, so two subsequent mini-batches of examples should also be independent from each other.



Note that $\mathbf{w} = [w_0, w_1, \cdots, w_m]$

Algorithm 2: Stochastic Gradient Decent

Input: Training data $\mathcal{D}$, Learning rate schedule
Result: Final Weights $\mathbf{w}$
Initialize weights randomly;
$t \leftarrow 1$;
while *not converged* **do**
  Sample an iid batch from training data, $\mathcal{D}_b \subset \mathcal{D}$;
  Compute Gradients using $\mathcal{D}_b$, $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$;
  Update weights, $\mathbf{w} \leftarrow \mathbf{w} - \alpha_t \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$;
  $t \leftarrow t + 1$;
  Check for convergence ;
end

α_t is the learning rate at iteration t.

Implementation - PyTorch

An Epoch is one pass through all the training data.

The data should be **shuffled** after each epoch to make sure the gradients are independent. This should be handled in the data generator.

More in labs.

```python
import torch
import torch.optim as optim

# The optimizer directly takes the model's parameters and the learning rate
optimizer = optim.SGD(model.parameters(), lr=0.001)

EPOCHS = 5
for epoch in range(EPOCHS):
    running_loss = 0.0
    num_batches = 0

    # Iterate over all batches (usually a DataLoader in PyTorch)
    for image_batch, label_batch in train_loader:
        # 1. Clear gradients from the previous step
        optimizer.zero_grad()
        # 2. Forward pass
        predictions = model(image_batch)
        # 3. Compute loss
        loss = loss_fn(predictions, label_batch)
        # 4. Backward pass (compute gradients for all model parameters)
        loss.backward()
        # 5. Update weights based on the computed gradients
        optimizer.step()
        # Track the loss (using .item() pulls the scalar value out of the tensor)
        running_loss += loss.item()
        num_batches += 1

    # Calculate and print the mean loss for the epoch
    epoch_loss = running_loss / num_batches
```

Aside: Learning Curves

Learning curves are a powerful tool to understand the behaviour of NN training.

More on plotting learning curve in labs.



Image: Standford cs231n

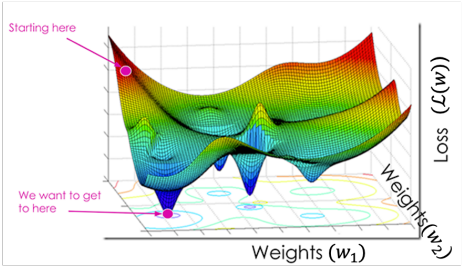
Revision Questions

1. What are the advantages of using a small batch size?
2. Why do we need to shuffle data between epoch?
3. Why is there a negative sign in the weight update of the SGD?

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

Local Minima



With **non-convex** functions, such as neural nets, it is possible to have many local minima.

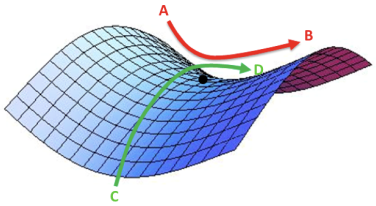
Local minima can be problematic if they have high cost in comparison to the global minimum.

"For large neural networks, most local minima have a low cost value, and that it is not important to find a true global minimum rather than to find weights with low cost."

To rule out local minima: plot the norm of the gradient over time.

- If the norm of the gradient does not shrink, the problem is not any kind of critical point.
- Positively establishing that local minima are the problem can be very difficult in DNN.

Saddle Points



A saddle point has a local minimum along one cross-section of the cost function and a local maximum along another cross-section.

- In higher-dimensional spaces, local minima are rare, and saddle points are more common.
- For first-order optimization, the gradient can often become very small near a saddle point/Flat Regions.
- Second-order methods (Newton's method) can terminate at saddle points.

Flat Regions, Cliffs and Exploding Gradients

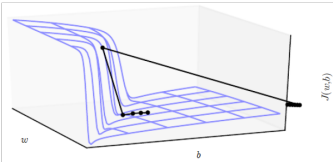


Image: Goodfellow 2016

- Neural networks with many layers often have extremely steep regions resembling cliffs.
- On the face of an extremely steep cliff structure, the gradient update step can move the parameters extremely far.
- Serious consequences can be avoided using the gradient clipping. This prevents any gradient to have norm greater than a threshold.

Vanishing and **Exploding** gradient problem.

Arises when

- The computational graph becomes extremely deep.
- Repeatedly applying the same operation at each time step of a long temporal sequence (in RNN discussed later)

Vanishing gradients make it difficult to know which direction the parameters should move to improve the cost function, while exploding gradients can make learning unstable.

How can we solve these problems?

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

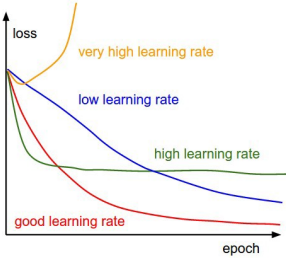


Image: Standford cs231n

We already discussed SGD.
The main parameter is learning rate. This can be a fixed value (α) or can vary in time. A simple linear learning rate schedule is below:

$$\alpha_t = \left(1 - \frac{t}{\tau}\right)\alpha_0 + \frac{t}{\tau}\alpha_\tau$$

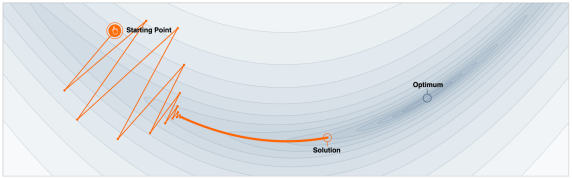
α_0 initial learning rate, α_τ final learning rate, τ number of iterations.

The learning rate may be chosen by trial and error. Best to choose it by monitoring learning curves.

- If it is too large, the learning curve will show violent oscillations,and often increasing significantly.
- If the learning rate is too low, learning proceeds slowly.

Normal gradient decent update:

$$\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} - \alpha_t \, \nabla_{\mathbf{w}} \mathcal{L}\left(\mathbf{w}\right)$$



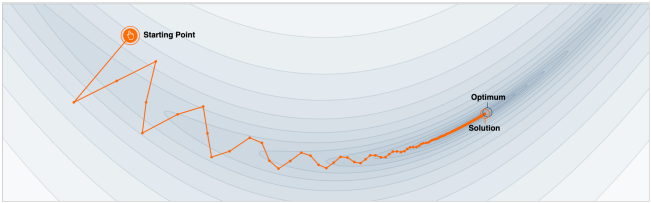
Pathological curvature: regions of cost function which are not scaled properly.

The iterates either jump between valleys, or approach the optimum in small steps.

Momentum give gradient descent a short-term memory.

$$\mathbf{z}^{[t]} \leftarrow \beta \mathbf{z}^{[t-1]} - (1-\beta)\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w})$$
$$\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} + \alpha \mathbf{z}^{[t]}$$

Hyper-parameter β controls how quickly the contributions of previous gradients exponentially decay. A typical value is $\beta = 0.9$.

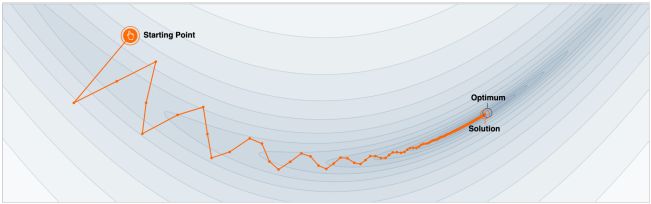


Why Momentum Really Works
On the importance of initialization and momentum in deep learning.

Variants: Nesterov momentum. The gradient is evaluated after the current velocity is applied.

$$\mathbf{z}^{[t]} \leftarrow \beta \mathbf{z}^{[t-1]} - (1-\beta)\nabla_{\mathbf{w}}\mathcal{L}\left(\mathbf{w} + \beta \mathbf{z}^{[t-1]}\right)$$
$$\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} + \alpha_t \mathbf{z}^{[t]}$$

Hyper-parameter β controls how quickly the contributions of previous gradients exponentially decay.



Why Momentum Really Works
On the importance of initialization and momentum in deep learning.

- How can the problem of saddle points be minimized first-order methods?
- What happens if $\beta = 0$ in momentum gradient descent.
- Why is it a good idea to reduce the learning rate with iterations in SGD?

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

Adaptive Learning Rate

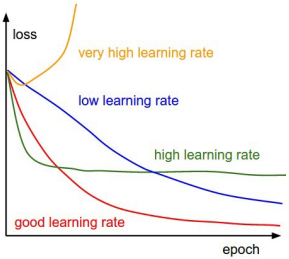


Image: Standford cs231n

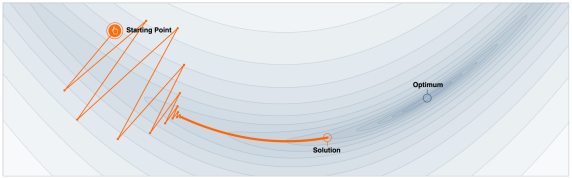
Learning rate is a key parameter in gradient descent.

- If the learning rate is too small, updates are small and optimization is slow.
- If the learning rate is too large, updates will be large and the optimization is likely to diverge.

Start with a large learning rate and **decay** it during training.

Finding the "best decay schedule" is not trivial. **Adaptive learning-rate algorithms** such as Adam and RMSprop can adjust the learning rate during the optimization process.

Normal gradient decent update: $\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} - \alpha_t \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$



Can have oscillation: Slow in one direction and speed up in other. RMSprop use the root mean square of the gradient in each direction to scale the update.

$$r^{[t]} = \rho r^{[t-1]} + (1-\rho)\left(\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})\right)^2$$

$$\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} - \alpha \frac{\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})}{\sqrt{r^{[t]} + \epsilon}}$$

ϵ is a small value to prevent division by zero ($\epsilon = 10^{-8}$). $r^{[t]}$ is a vector. Division and square applied element-wise

Combines RMSprop and momentum. ADAM: A Method For Stochastic Optimization

$$r^{[t]} = \rho r^{[t-1]} + (1-\rho)\left(\nabla_{\mathbf{w}}\mathcal{L}\left(\mathbf{w}\right)\right)^2 \quad \triangleright \text{RMSprop}$$

$$\mathbf{z}^{[t]} \leftarrow \beta \mathbf{z}^{[t-1]} + (1-\beta)\nabla_{\mathbf{w}}\mathcal{L}\left(\mathbf{w}\right) \quad \triangleright \text{Momentum}$$

$$r^{[t]} \leftarrow r^{[t]}/(1-\rho)\ ;\ \mathbf{z}^{[t]} \leftarrow \mathbf{z}^{[t]}/(1-\beta) \quad \triangleright \text{Bias correction}$$

$$\mathbf{w}^{[t]} \leftarrow \mathbf{w}^{[t-1]} - \alpha \frac{\mathbf{z}^{[t]}}{\sqrt{r^{[t]} + \epsilon}}$$

ϵ is a small value to prevent division by zero ($\epsilon = 10^{-8}$). $s^{[t]}$ is a vector. Division and square applied element-wise

Works with wide range of problems. Hyper parameters $[\beta, \rho]$ are named $[\beta_1, \beta_2]$ in pytorch and usually are set to $\beta_1 = 0.9, \beta_2 = 0.999$.

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

Choosing the Right Optimization Algorithm

Unfortunately, there is currently no consensus on which algorithm to use.

SGD	Gradient descent can use parallelization efficiently. SGD usually converges faster than GD on large datasets. Of the optimizers profiled here, SGD uses the least memory for given batch size.
Momentum	Momentum usually speeds up the learning. Momentum uses more memory for a given batch size than stochastic gradient descent but less than RMSprop and Adam.
RMSprop	RMSprop's adaptive learning rate usually prevents the learning rate decay from diminishing too slowly or too fast. RMSprop maintains per-parameter learning rates.
Adam	The hyperparameters of Adam are usually set to predefined values. Adam performs a form of learning rate annealing with adaptive step-sizes. Adam is often the default optimizer in machine learning.

If your input data is sparse, then you likely achieve the best results using one of the adaptive learning-rate methods.

An overview of gradient descent optimization algorithms

Beyond Gradient Descent

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

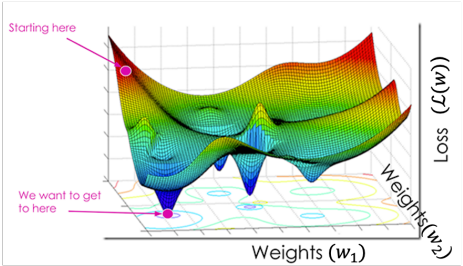
Deep networks has many parameters:

$$h(\mathbf{x}) = h^{(3)}\left(h^{(2)}\left(h^{(1)}\left(\mathbf{x}; \mathbf{W}^{(1)}\right); \mathbf{W}^{(2)}\right); \mathbf{W}^{(3)}\right)$$

Gradient descent learning is iterative and thus require the user to specify some initial point.

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \alpha_t \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

Starting place of gradient descent can be critical to the model's ultimate performance.



The initial point can determine whether the algorithm converges:

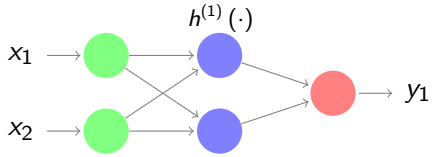
- Some initial points can be unstable and the algorithm may fails altogether.
- Initial point can determine how quickly learning converges.
- local minimums can have wildly varying generalization error, and the initial point can affect which minimum is selected.

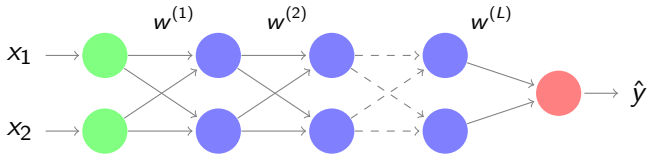
The most important consideration when initializing the weights is that the **initial parameters need to "break symmetry" between different units**.

Initial parameters need to "break symmetry" between different units.

- If two hidden units with the same activation function are connected to the same inputs, then these units must have different initial parameters.
- If they have the same initial parameters, then a learning algorithm will constantly update both of these units in the same way.

This motivates random initialization of the parameters.





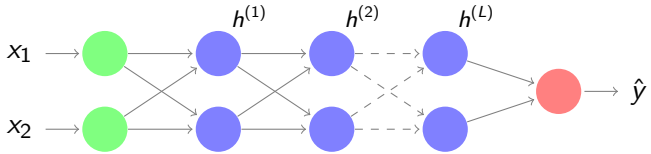
Can initialize all the weights in the model to values drawn randomly from a Gaussian or uniform distribution.

The scale of the initial distribution, have a large effect on the outcome of the optimization procedure.

- **Too small:** vanishing gradients, learning too slow.
- **Too large:** exploding gradients, unstable.

Assuming all activation's are linear.

$$\hat{y} = w^{(L)} w^{(L-1)} \dots w^{(2)} w^{(1)} x$$



Rule of thumb to prevent exploding/vanishing gradients:

- The mean of the activations should be zero.

$$\mathbb{E}\left(h^{(l-1)}\right) = \mathbb{E}\left(h^{(l)}\right) = 0$$

- The variance of the activations should stay the same across every layer.

$$\mathrm{Var}\left(h^{(l-1)}\right) = \mathrm{Var}\left(h^{(l)}\right)$$

All the weights of layer ll are picked randomly from a normal distribution with mean $\mu = 0$ and variance $\sigma^2 = \frac{1}{n^{[l-1]}}$ where $n^{[l-1]}$ is the number of neuron in layer $l-1$.

$$\mathbf{W}^{(l)} \sim \mathcal{N}\left(0, \frac{1}{n^{[l-1]}}\right)$$

Biases are initialized with zeros.

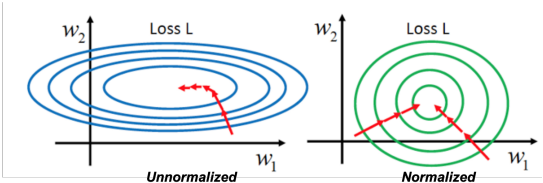
$$\mathbf{b}^{(l)} = 0$$

The mathematics of why this works is explained in Deeplearning.ai notes
A varient of Xavier, that works well with "ReLU" is provided in He normalization

Outline

1. Loss Function & Empirical Risk Minimization
2. Back-Propagation
3. Stochastic Mini-batch Gradient Descent
4. Challenges in Neural Network Optimization
5. Basic Algorithms
6. Advanced Algorithms: Adaptive Learning Rate
7. Choosing the Right Optimization Algorithm
8. Parameter Initialization Strategies
9. Batch Normalization

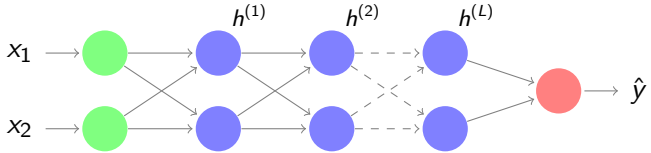
Generally we know that normalizing the input features help to speedup learning.



$$\mu_j = \frac{1}{N}\sum_i x_j^{(i)}; \quad \sigma_j^2 = \frac{1}{N}\sum_i \left(x_j^{(i)} - \mu_j\right)^2$$

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

How about hidden layer activations?



Can we normalize input to each layer?

$$\mathbf{z}^{(i)} = \mathbf{W}^{(l)}\mathbf{x}^{(i)} + \mathbf{b}^{(l)} \quad \triangleright \text{Affine transform}$$
$$h^{(i)} = g\left(\mathbf{z}^{(i)}\right) \quad \triangleright \text{Activation}$$

Typically we normalize $\mathbf{z}^{(i)}$.

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

$$\mu_j = \frac{1}{m}\sum_i z_j^{(i)}; \quad \sigma_j^2 = \frac{1}{m}\sum_i \left(z_j^{(i)} - \mu_j\right)^2$$

$$\hat{z}_j^{(i)} = \frac{z_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \triangleright \text{mean zero, variance 1}$$

$$\tilde{z}_j^{(i)} = \gamma \hat{z}_j^{(i)} + \beta \quad \triangleright \gamma, \beta \text{ learnable parameters}$$

Batch Normalization allows us to normalize the feature values across a mini-batch.

The mean and variance is computed over the mini-batch.

The bias in affine transform is not useful when using batch-norm:
$\mathbf{z}^{(i)} = \mathbf{W}^{(l)}\mathbf{x}^{(i)} + \cancel{\mathbf{b}^{(l)}}$

At test time you do not have a batch to compute mean and variance.

Keep a weighted average over mini-batches while training.

$$\hat{\mu}_j = \rho\ \hat{\mu}_j + (1-\rho)\,\mu_j$$
$$\hat{\sigma}_j^2 = \rho\ \hat{\sigma}_j^2 + (1-\rho)\,\sigma_j^2$$

Apply the moving averages while testing.

$$\hat{z}_j^{(i)} = \frac{z_j^{(i)} - \hat{\mu}_j}{\sqrt{\hat{\sigma}_j^2 + \epsilon}}$$

$$\tilde{z}_j^{(i)} = \gamma \hat{z}_j^{(i)} + \beta$$

Revision Questions

1. What happens if the weights in a neural network are all initialized to 0?
2. Why should we not have a bias in neural network layers when batch-norm is used?
3. Should batch-norm be used before or after the activation?

Summary

1. Neural networks are usually trained using iterative, gradient based methods.
2. There are many gradient based optimization techniques. Methods with adaptive learning rate seems to be robust to many problems.
3. Initialization and batch normalization help speed up learning.

Next week:

1. Convolutions